UME

Organisation / Business Operating System

Comprehensive Design & Architecture Document

Document ID: UME-DA-002

Version: 2.0.0 | Status: Updated — OrgOS Factory Integration

Date: March 2026

Architecture Style: OrgOS Factory · Kernel-Centric · Event-Driven · DAO-Capable

Scope: OrgOS Factory · OrgKernel · OrgApps · OrgExec/ODD · SDO · DAO · DLT · OrgDNA

Document Change History

Version	Date	Author	Status	Summary
1.0.0	Jan 2026	UME Architecture Team	Approved	Baseline: Kernel + 42 modules + device drivers
2.0.0	Mar 2026	UME Architecture Team	Updated	OrgOS Factory model · OrgApps · OrgExec/OrgCPU · ODD · SDO/Digital Twin · OrgDNA · DAO · DLT

Summary of Changes in v2.0.0

Version 2.0.0 introduces the OrgOS Factory model — a fundamental reframing of the UME platform from a single Business OS to a factory that produces configurable Organisation Operating Systems (OrgOS) for any organisation at any scale. All v1.0.0 content remains valid and is extended by these additions.

Change Area	v1.0.0 Concept	v2.0.0 Updated Concept	Impact
Platform Model	UME is a Business OS platform	UME is an OrgOS Factory — a meta-OS that produces and manages OrgOS instances for any organisation	Fundamental: changes the positioning and architecture narrative throughout
Module Layer	Organisation Modules (KernelModule trait)	OrgApps (OrgApp trait extending KernelModule) — user-facing applications managing an organisation	Backward-compatible: OrgApp extends KernelModule; all existing modules remain valid
Device Drivers	KernelDeviceDriver (external system adapters)	Dual model: (1) KernelDeviceDriver for external systems; (2) ODD (OrgDeviceDriver) for actors (humans/AI/devices)	Additive: ODD is new; existing DeviceDriver pattern unchanged
Actor Model	Users and external systems	Unified actor model: OrgExecs (humans/AI/hybrid) + OrgCPUs (machines/IoT), all governed by ODD contracts	New: no prior equivalent; replaces implicit user model with explicit actor governance
Organisation State	Org data in modules	SDO — Software Defined Organisation as live digital twin of the org	Additive: SDO is new aggregate view; does not change individual OrgApp data models
Configuration	Static KernelConfig	OrgDNA — versioned, exportable, replicable master config package	Additive: OrgDNA wraps existing config; enables cloning and franchising
Multi-Org	Not supported	DAO — network of OrgOS instances under shared smart contract governance on DLT	New: DAO and DLT are new modules; single-org deployments unchanged
Organisational Memory	Audit chain (within single org)	OrgMemory = local audit chain (single org) + DLT (cross-org DAO)	Additive: DLT extends the existing audit chain concept to inter-org scope

Part I: System Overview & Design Principles

Updated in v2.0.0: OrgOS Factory Model

UME is a software-defined Organisation Operating System (OrgOS) factory built in Rust. It models every organisation as a kernel-managed collection of domain applications (OrgApps) executed by a unified actor model (OrgExecs + OrgCPUs) governed by OrgDeviceDriver (ODD) contracts — analogous to an operating system where a kernel manages hardware through device drivers and runs applications for users.

1.1 The OrgOS Factory Model

The central architectural insight of v2.0.0 is that UME is not one operating system — it is a factory that produces, deploys, and manages operating systems for organisations. Just as a cloud provider provisions virtual machines on demand, UME provisions Organisation Operating Systems on demand.

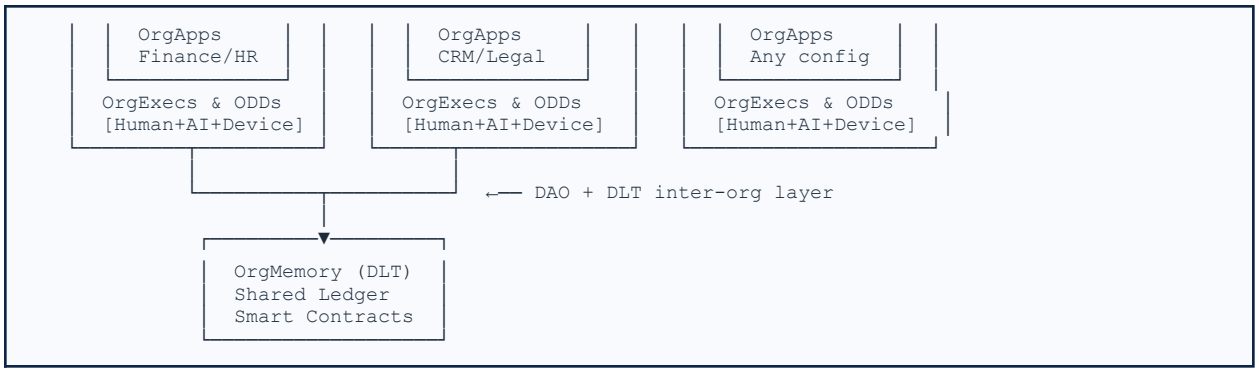
Layer	v1.0.0 Concept	v2.0.0 Concept
Meta-system	UME Platform	OrgOS Factory — produces OrgOS instances on demand for any organisation
Per-org OS	Single UME instance	OrgOS — a configured operating system owned by and running for one specific organisation
Runtime core	UME Kernel	OrgKernel — the kernel of a specific OrgOS instance
Applications	Org Modules	OrgApps — the configurable applications running inside an OrgOS
Actor model	Users + external systems	OrgExecs (humans/AI/hybrid) + OrgCPUs (machines/IoT) governed by ODD contracts
Config package	Static KernelConfig	OrgDNA — versioned, exportable master configuration of an entire OrgOS
Org representation	Data in modules	SDO — Software Defined Organisation, the live digital twin of the organisation
Multi-org network	Not supported	DAO — network of OrgOS instances with smart contract governance on shared DLT

OrgOS Factory — System Context Diagram (C4 Level 0)

The UME platform acts as an OrgOS Factory: a meta-operating-system that produces, configures, and maintains Organisation Operating Systems for any organisation.

```
UME PLATFORM  (OrgOS Factory)

┌──────────────────────────────────────────────────────────────┐
│                        UME META-KERNEL                       │
│  OrgOS Provisioner │ OrgDNA Manager │ Template Gallery │ Marketplace │
└──────────────────────────────────────────────────────────────┘
        │                                │
        ▼                                ▼
┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│ OrgOS Instance A │   │ OrgOS Instance B │   │ OrgOS Instance N │
│ Bob's Dry Cleaning│   │ Alice Consulting │   │ Any Organisation │
│                  │   │                  │   │                  │
│ OrgKernel        │   │ OrgKernel        │   │ OrgKernel        │
│ ┌──────────────┐ │   │ ┌──────────────┐ │   │ ┌──────────────┐ │
```



1.2 Core Design Principles (Updated)

Principle	Statement	Consequence
OrgOS Factory	UME provisions, manages, and maintains OrgOS instances; it does not impose a single runtime on all organisations	Each org gets an isolated, configurable OrgOS; factory handles lifecycle
Kernel-Centric Supervision	The OrgKernel is the single composition root for all OrgApps, OrgExecs, and OrgCPUs within an OrgOS	All resources, permissions, and events flow through the kernel
Uniform Actor Model	All executors of work — humans, AI agents, machines — are modelled uniformly as OrgExec/OrgCPU with ODD contracts	Single governance model for all actors; no special-casing for AI vs. human
SDO as Ground Truth	The Software Defined Organisation is the single source of truth for all organisational state	No data lives outside the SDO; all integrations are projections of the SDO
OrgDNA Replicability	Every OrgOS can be fully described, exported, and replicated via OrgDNA	Enables franchising, cloning, templating, and DAO coordination
DAO-Native	Multiple OrgOS instances can form DAOs via smart contracts on a shared DLT	Horizontal and vertical organisation scaling without platform migration
Dependency Inversion	OrgApps depend only on ume_core contracts; never on the OrgKernel or other OrgApps	Domain isolation; independent testability and replaceability
Event-Driven Integration	All cross-OrgApp communication is via the event bus; no direct OrgApp-to-OrgApp calls	Loose coupling; OrgApps are independently stoppable and replaceable
RBAC at Every Boundary	All access to data and operations is governed by the RBAC engine; ODDs declare permissions	Consistent security; no bypass paths; AI agents subject to same controls as humans
Immutable Audit	All actions — by humans, AI, and machines — produce immutable audit records	Non-repudiation; compliance-grade audit trail from day one

Part II: OrgKernel Architecture

The OrgKernel is the per-organisation kernel: the composition root, resource manager, supervisor, and event hub for a single OrgOS instance. Each OrgOS has exactly one OrgKernel. Multiple OrgOS instances may run on the same infrastructure, each with an isolated OrgKernel.

2.1 OrgKernel Core Structure

OrgKernel (ume_kernel::kernel)

```
pub struct OrgKernel {
    // Identity
    org_id:       OrgId,
    org_dna:      Arc<RwLock<OrgDNA>>,
    // Original kernel subsystems (v1.0.0)
    config:       KernelConfig,
    registry:     ModuleRegistry,     // now: OrgAppRegistry
    device_bus:   DeviceBus,
    executor_pool:ExecutorPool,
    event_bus:    InMemoryEventBus,
    rbac:         RbacEngine,
    memory:       MemoryManager,
    storage:      StorageManager,
    network:      NetworkManager,
    log_audit:    LogAuditManager,
    supervisor:   SupervisorEngine,
    orchestrator: OrchestratorEngine,
    // New v2.0.0 subsystems
    exec_registry:OrgExecRegistry,    // all OrgExecs + ODDs
    cpu_registry: OrgCPURegistry,     // all OrgCPUs + ODDs
    odd_bus:      ODDBus,             // ODD validation + enforcement
    sdo:          SDOEngine,          // digital twin engine
    dao_client:   Option<DAOClient>,  // present if DAO member
    dlt_client:   Option<DLTClient>,  // present if DAO member
}
```

2.2 Extended Boot Sequence (v2.0.0)

Step	Action	New in v2.0.0
1	Parse and validate KernelConfig + OrgDNA from persistent store	OrgDNA load added
2	Initialize LogAuditManager — all subsequent boot steps are logged	—
3	Initialize MemoryManager — region allocators and cache layers	—
4	Initialize StorageManager — open configured backend adapters	—
5	Initialize NetworkManager — bind internal event transport channels	—

6	Initialize RbacEngine — load role/permission definitions from OrgDNA	OrgDNA source added
7	Initialize DeviceBus — enumerate and register kernel device drivers	—
8	Initialize ExecutorPool — spawn executor threads per OrgDNA config	OrgDNA source added
9	Initialize InMemoryEventBus (or configured broker)	—
10	Initialize SupervisorEngine and OrchestratorEngine	—
11	Load OrgAppRegistry — instantiate and register all enabled OrgApps from OrgDNA	OrgApp name; OrgDNA source
12	Initialize ODDBus — load all ODD contracts for OrgExecs and OrgCPUs	New v2.0.0
13	Initialize OrgExecRegistry and OrgCPURegistry — register all actors	New v2.0.0
14	Initialize SDOEngine — build initial digital twin from current OrgApp state	New v2.0.0
15	Bootstrap DAO/DLT clients if this OrgOS is a DAO member	New v2.0.0
16	Run bootstrap_org() — activate default roles, OrgApps, and OrgExecs	—
17	Emit kernel.boot event; transition all subsystems to Running state	—
18	Start HTTP server (ume_server) and expose operational endpoints	—

2.3 ODD Bus (v2.0.0 New Subsystem)

The ODDBus is the new kernel subsystem responsible for managing, validating, and enforcing all OrgDeviceDriver contracts. Every action taken by any OrgExec or OrgCPU is validated against its ODD before execution.

ODDBus API	Behaviour
register_odd(odd: impl OrgDeviceDriver)	Register an ODD contract for an OrgExec or OrgCPU; validate contract completeness
validate_action(exec_id, action) -> OddDecision	Check whether an action is permitted by the actor's ODD; returns Allow | Deny | Escalate
get_odd(exec_id) -> ODD	Retrieve the current ODD contract for a given OrgExec or OrgCPU
update_odd(exec_id, new_odd)	Update an ODD contract; all in-flight actions complete under old contract
list_violations(exec_id, from, to)	Retrieve all ODD constraint violations for an actor in a time window
health_check_all()	Run health check on all registered ODD actors; flag degraded/offline OrgCPUs

Part III: OrgApps — Organisation Module Layer

OrgApps are the domain applications of an OrgOS. Formerly called "Organisation Modules", OrgApps implement all business domain capabilities. The OrgApp concept extends the KernelModule protocol with marketplace metadata, OrgDNA serialisation, and configuration wizard support.

3.1 OrgApp Contract

Every OrgApp MUST implement the OrgApp trait (which extends KernelModule). The full module contract from v1.0.0 remains intact. New requirements in v2.0.0:

Requirement	v1.0.0	v2.0.0 Addition
Identity	id, name, domain, version	+ app_id (marketplace), app_category, app_tier
Lifecycle	start, stop, pause, resume, health_check	No change
OrgDNA Integration	Not present	to_orgdna_component(), from_orgdna_component() — for OrgDNA export/import
Marketplace	Not present	marketplace_metadata() — icon, description, dependencies, pricing tier, screenshots
Configuration	Static KernelConfig	config_schema() — JSON Schema driving the UI configuration wizard in the OrgOS installer
Event Contract	Emit domain events	No change; events also update SDO state via SDOEngine subscription
RBAC	Declare permissions	ODD integration: capability grants to OrgExecs/OrgCPUs are now via ODD, not direct RBAC only

3.2 OrgApp Tiers

Tier	Availability	Description
Core	All OrgOS instances (always on)	OrgKernel, Dashboard, User Management, OrgDNA Manager, SDO Engine
Starter	Starter plan and above	Finance, HR, Legal & Compliance, Basic CRM, Work Management
Growth	Growth plan and above	Risk & GRC, Sales, Operations/ERP, Marketing, OrgExec AI Agents
Enterprise	Enterprise plan only	DAO & DLT, Advanced Analytics, Custom OrgApps, Multi-entity consolidation
Marketplace	Any plan (add-on pricing)	Third-party integrations, industry-specific extensions, custom OrgApps via SDK

3.3 Complete OrgApp Index (42 built-in + new v2.0.0)

#	OrgApp Name	Domain	Tier	v2.0.0 Changes
01	Organisation Administration	Administration	Core	OrgDNA integration; org unit tree syncs to SDO topology

02	Organisation Analytics	Analytics	Growth	SDO state feeds analytics; OrgSimulator integration
03	Backup, Recovery & State Management	Resilience	Core	OrgDNA backup added to backup scope
04	Board Management	Governance	Growth	DAO governance votes can originate from Board module
05	Business Development & Analysis	Growth	Growth	OrgSimulator integration for deal scenario modelling
06	Enterprise Content Management	Content	Growth	Document vault stores ODD contracts and OrgDNA snapshots
07	Organisation Communications	Communications	Starter	Inter-org comms via DAO event bus for DAO members
08	Client & Stakeholder CRM	Relationships	Starter	Customer OrgExec type: customers as external actors with read ODDs
09	Organisation Design System	Design	Core	OrgDNA includes design tokens and brand assets
10	Engineering & Technology	Engineering	Growth	IoT OrgCPU onboarding wizard; ODD templates for devices
11	ESG, CSR & Sustainability	Sustainability	Growth	SDO includes ESG metrics layer
12	Enterprise Engineering & Admin	Enterprise Mgmt	Enterprise	Multi-OrgOS management console for enterprise groups
13	Legal Entity Management (Chombo)	Legal	Starter	Entity data flows to OrgDNA jurisdiction packs
14	Finance & Accounting	Finance	Starter	Financial events feed DLT for inter-org royalty/payment contracts
15	Governance, Risk & Compliance	GRC	Growth	KRI thresholds from OrgDNA; cross-DAO risk reporting
16	Human Resources	People	Starter	OrgExec onboarding wizard; ODD generation for new hires
17	Investment Management	Investment	Enterprise	Investment funds modelled as separate OrgOS instances in a DAO
18	IT & Asset Management	IT	Growth	OrgCPU registry integration; device ODD management
19	Enterprise Knowledge Management	Knowledge	Growth	OrgMemory search interface; DLT document anchoring
20	Learning & Development	Learning	Growth	ODD skill/competency requirements for OrgExec assignments
21	Management & Strategy	Strategy	Growth	SDO OrgSimulator integration for strategic scenarios
22	Marketing System (Soko)	Marketing	Growth	OrgDNA brand pack drives marketing assets
23	Master Data Management	Data	Core	OrgDNA is the master data anchor; MDM syncs to OrgDNA
24	Office & Facility Management	Facilities	Growth	Physical location OrgCPUs: HVAC, access control, IoT sensors

25	Operations Management	Operations	Starter	OrgCPU (machine) status dashboard; IoT ODD management
26	Portal, Hub & Dashboard	Portal	Core	SDO state drives executive dashboard; employee portal ODD-scoped views
27	Portfolio & Program Management	Portfolio	Enterprise	Portfolio of OrgOS instances in a holding DAO
28	PR & Branding System	Brand	Growth	OrgDNA brand package management; DAO brand standards enforcement
29	Process, Orchestration & Workflow	Process	Starter	ODD-aware workflow: steps assigned to OrgExec types
30	Product, Services & Solution Mgmt	Product	Growth	Product catalogue included in OrgDNA for franchise deployments
31	Production, Manufacturing & Fabrication	Production	Enterprise	Heavy OrgCPU (manufacturing) ODD management
32	Requirements Management	Requirements	Growth	ODD capability requirements for new OrgExec/OrgCPU onboarding
33	Enterprise Risk Management	Risk	Growth	KRI monitoring feeds SDO risk layer; DLT risk attestations
34	Sales Management	Sales	Starter	Customer orders in CRM linked to OrgCPU fulfilment tracking
35	Enterprise Schedule Management	Schedule	Growth	AI OrgExec (scheduling agent) ODD-managed optimisation
36	Security, Privacy & Protection	Security	Core	ODD enforcement engine; AI agent action audit trail
37	Logistics, Supply Chain & Warehouse	Supply Chain	Growth	Vendor orgs as external OrgOS instances in a supply chain DAO
38	Team & Cooperative Management	Teams	Growth	Cooperative ownership modelled as multi-member DAO
39	Organisation Templating System	Templates	Core	OrgDNA template library; public/private template marketplace
40	Enterprise Work Management	Work	Starter	OrgExec task assignment via ODD capability matching
41	Custom UME Modules	Extension	Marketplace	SDK v2 adds ODD integration requirements for custom OrgApps
42	Custom Organisation Modules	Extension	Marketplace	Third-party OrgApps must implement OrgApp trait + ODD integration
43 (new)	OrgExec & ODD Manager	OrgOS Meta	Core	New in v2.0.0: manage all OrgExecs, OrgCPUs, and their ODD contracts
44 (new)	DAO & Distributed Ledger	Inter-Org	Enterprise	New in v2.0.0: form DAOs, manage smart contracts, view DLT ledger
45 (new)	Digital Twin (SDO) Engine	OrgOS Meta	Core	New in v2.0.0: live org topology, SDO state, and OrgSimulator

46 (new)	OrgDNA Manager	OrgOS Meta	Core	New in v2.0.0: export, import, version, and broadcast OrgDNA packages

Part IV: Actor Model — OrgExec, OrgCPU & ODD

OrgExec, OrgCPU & ODD Architecture

Every actor in an OrgOS — whether human, AI, machine, or device — is modelled as an OrgExec or OrgCPU, governed by an OrgDeviceDriver (ODD) contract. This uniform actor model enables the OrgKernel to plan, route, audit, and govern all work execution through a single standardised interface, regardless of the nature of the executor.

OrgExec Classification

Type	Description	Examples	ODD Category
Human OrgExec	Natural persons: employees, contractors, founders, consultants operating within the OrgOS.	CEO, Accountant, HR Manager, Customer Service Rep, Contractor	FTE-ODD, CONTRACTOR-ODD , EXEC-ODD
AI Agent OrgExec	LLM-powered or rule-based autonomous agents that execute tasks within defined ODD boundaries without human intervention per task.	Scheduling AI, Customer Comms Agent, Compliance Monitor, Demand Forecaster	AI-AGENT-ODD, LLM-ODD
Hybrid OrgExec	Human-AI collaborative systems where the AI handles task decomposition/suggestion and a human provides final approval or judgment.	AI-assisted underwriting, AI draft + human review workflows	HYBRID-ODD
OrgCPU	Digital-only executors: machines, IoT devices, sensors, 3rd-party platforms, manufacturing equipment, payment terminals.	CNC Machine, POS Terminal, Environmental Sensor, ERP System, Payment Processor	IOT-ODD, MACHINE-ODD, PLATFORM-ODD

ODD (OrgDeviceDriver) — Contract Structure

An ODD is the binding contract that governs how a specific OrgExec or OrgCPU interacts with the OrgKernel and all other system components. It is the organisational equivalent of an operating system device driver — standardising the interface between the actor and the system.

ODD Section	Purpose	Examples
Identity	Who or what this OrgExec/OrgCPU is: unique ID, display name, type, version, responsible owner.	EMP-0009, AI-SCHED-01, WASHER-01
Capabilities	Explicit list of what this actor CAN do: read data, write data, trigger workflows, send notifications, make payments.	Read: order queue; Write: schedule assignments; Notify: staff via SMS
Constraints	Explicit list of what this actor MUST NOT do or MUST do within limits: rate limits, working hours, approval thresholds, data scopes.	Max 200 executions/hour; working hours only; approval required for changes affecting >3 staff
Data Access	Granular data access rights: which OrgApps, which records, which fields, read vs. write vs. admin.	Read: HR.employees (own department); Write: Ops.orders (assigned only)

Escalation Rules	Conditions under which this actor must pause and escalate to a human or higher-authority OrgExec.	Escalate if: conflicting rules cannot be resolved; error rate > 5%; confidence < 70%
Audit Requirements	How all actions by this actor are logged: log level, retention period, real-time vs. batch.	All writes logged; 12-month retention; real-time SIEM feed for AI agents
Performance Contract	Expected throughput, latency SLA, and resource consumption limits.	Process 50 orders/min; respond in <200ms; max 10% CPU on org.ops executor
Lifecycle	How this actor is started, paused, stopped, and restarted. Graceful shutdown requirements.	Graceful drain: complete in-flight tasks before stop; no hard kill

ODD Rust Interface

ODD Trait (ume_core::odd)

```
pub trait OrgDeviceDriver: Send + Sync {
    fn odd_id(&self) -> &'static str;
    fn odd_version(&self) -> &'static str;
    fn exec_type(&self) -> OrgExecType;  // Human | AIAgent | Hybrid | OrgCPU
    fn capabilities(&self) -> Vec<OddCapability>;
    fn constraints(&self) -> Vec<OddConstraint>;
    fn data_access(&self) -> Vec<DataAccessGrant>;
    fn escalation_rules(&self) -> Vec<EscalationRule>;
    fn audit_config(&self) -> AuditConfig;
    fn lifecycle_policy(&self) -> LifecyclePolicy;
    fn health_check(&self) -> OddHealthReport;
    fn validate_action(&self, action: &OrgAction) -> OddDecision;  // Allow | Deny | Escalate
}
```

Part V: Software Defined Organisation (SDO) & OrgDNA

Software Defined Organisation (SDO) — Digital Twin Architecture

The Software Defined Organisation (SDO) is the digital twin of a physical organisation. Every organisation running on the UME platform is represented as an SDO: a live, real-time model of the organisation's structure, processes, data flows, OrgExecs, OrgCPUs, and operational state.

SDO Design Philosophy

An SDO is not a static representation — it is a live, continuously updated digital twin.
Every OrgApp event, every OrgExec action, every OrgCPU reading updates the SDO state in real time.
The SDO provides: (1) single source of ground truth for all organisational data;
(2) simulation and what-if analysis capabilities; (3) optimisation through AI/ML;
(4) OrgDNA export for cloning, franchising, and scaling.

SDO Components

SDO Component	Description	Technical Realisation
OrgTopology	The live graph of all OrgApps, OrgExecs, OrgCPUs, and their relationships and data flows.	Graph data structure maintained by OrgKernel; updated via event bus subscriptions
OrgState	The current operational state of the organisation: all active records, KPIs, risk indicators, and pending tasks across all OrgApps.	Aggregated read model maintained by SDO engine; derived from OrgApp event streams
OrgHistory	The immutable record of all organisational events in chronological order: the audit chain.	Append-only audit log; queryable via time range and domain filters
OrgMemory	The persistent, long-term organisational knowledge store: decisions, contracts, filings, resolutions. For DAOs, this extends to the DLT shared ledger.	Local: audit chain + document vault. Cross-org: DLT with shared ledger
OrgSimulator	The what-if engine: run scenarios against the current SDO state to model the impact of decisions before making them.	LLM-powered simulation engine + deterministic financial/risk models
OrgOptimiser	AI/ML component that analyses the SDO state and suggests optimisations to processes, staffing, resource allocation, and compliance.	Pluggable OrgApp (AI Agents module); feeds suggestions to Exec Dashboard

OrgDNA — Master Configuration & Replication

OrgDNA is the exportable, versionable, and replicable master configuration package of a fully configured OrgOS. It encodes everything needed to reconstruct or clone an organisation's operating system, from the kernel configuration to the chart of accounts, compliance packs, ODD templates, and RBAC policies.

OrgDNA Package Contents

Component	Contents	Purpose
Kernel Config	OrgKernel configuration: org_id, mode, enabled OrgApps, executor pool config, storage backends.	Reproduces the exact OrgOS runtime configuration
OrgApp Registry	All active OrgApps with their versions and settings. Dependency graph.	Recreates the full module configuration
ODD Library	All ODD templates and instances: employee contracts, AI agent boundaries, device configs.	Recreates the actor governance model
RBAC Policy Pack	All roles, permissions, and assignments defined for this OrgOS.	Recreates the security and access control model
Chart of Accounts	Full COA hierarchy with account codes, types, and classifications.	Recreates the financial structure
Compliance Packs	All jurisdiction-specific compliance calendars, filing obligations, and policy templates.	Recreates the compliance posture for each jurisdiction
KPI Definitions	All configured KPIs, KRI thresholds, and dashboard layouts.	Recreates the measurement and monitoring framework
Workflow Templates	All configured business processes, approval chains, and automation rules.	Recreates operational workflows
OrgOS Template Version	The version tag of the original OrgOS template (if derived from one).	Enables traceability back to the source template

OrgDNA Use Cases

Use Case	Description	Mechanism
Clone Organisation	Spin up a new OrgOS instance that is an exact operational copy of an existing one. E.g., open a new branch office.	Export OrgDNA → provision new OrgOS → import OrgDNA → adjust org_id and entity details
Franchise Deployment	Franchise owner publishes a brand OrgDNA package; franchisees clone it and operate autonomously under the brand standards embedded in the DNA.	OrgDNA Publisher in DAO module; franchisee accepts OrgDNA; smart contract enforces brand standards version compliance
Template Marketplace	Contribute a well-configured OrgOS as a reusable template in the UME Template Gallery for other organisations to use.	Sanitise OrgDNA (remove org-specific data) → submit to marketplace → publish as industry template
OrgOS Upgrade	Apply updates to OrgDNA components (new RBAC policies, updated compliance packs) across one or many OrgOS instances.	DAO OrgDNA update broadcast; member orgs receive diff notification; accept/review/merge
Simulation Baseline	Export OrgDNA as the baseline state for running simulations in the OrgSimulator.	SDO Simulator uses OrgDNA as the initial state; runs what-if scenarios

Part VI: DAO & Distributed Ledger Architecture

DAO & Distributed Ledger (DLT) Architecture

The UME platform enables multiple OrgOS instances to interconnect into Decentralised Autonomous Organisations (DAOs). A DAO is a network of independent but cooperating organisations, each with their own OrgOS, that operate under a shared governance model enforced by smart contracts on a distributed ledger (DLT).

DAO Architecture

Component	Description	Technical Detail
DAO Registry	Maintains the list of all member OrgOS instances, their membership status, and their consensus node addresses.	Stored in DAO OrgApp; replicated to DLT genesis block
DLT Layer	The shared distributed ledger. All inter-organisation transactions, smart contract events, OrgDNA updates, and governance votes are recorded as immutable blocks.	Pluggable consensus: default PBFT for permissioned DAOs; configurable for public chain anchoring
Smart Contract Engine	Executes predetermined agreements automatically when trigger conditions are met. Built on the DLT layer.	Smart contracts defined in UME visual builder; compiled to bytecode; executed by SC Engine on DLT nodes
OrgDNA Broadcast	The parent org (or designated governance admin) can publish OrgDNA updates that all member OrgOS instances receive and can accept or review.	Event-driven push via DAO event bus; versioned diffs; multi-sig acceptance for mandatory updates
Inter-Org Event Bus	Extends the OrgKernel event bus across organisation boundaries, enabling cross-org event subscriptions and notifications.	Federated NATS or Kafka cluster; org-to-org topic namespacing; mTLS between OrgOS nodes
Governance Engine	Implements the voting model defined in the DAO governance smart contract. Manages proposals, voting periods, quorum checks, and execution of approved decisions.	On-chain voting state; off-chain proposal authoring; automatic execution on quorum reached
Treasury Manager	Manages the DAO shared treasury (if configured): balances, multi-sig withdrawals, investment policies, fund allocation rules.	Multi-sig wallet; automatic allocation rules from smart contracts; full DLT transaction history

DLT Transaction Types

Transaction Type	Trigger	Participants	Effect
ORGOS_JOIN	New org joins DAO	Joining org + DAO governance	Member added; OrgDNA snapshot stored; voting weight assigned
ROYALTY_PAYMENT	Smart contract trigger on revenue event	Franchisee org → Parent org	Automatic treasury transfer; royalty record created; DLT block committed
ORGDNA_UPDATE	Parent publishes OrgDNA update	Parent org → all members	Update broadcast; member orgs notified; acceptance tracking on-chain

GOVERNANCE_VOTE	Governance proposal submitted	All member orgs	Vote recorded; quorum tracked; automatic execution on threshold met
SMART_CONTRACT_EXEC	Any contract trigger condition met	Counterparty orgs	Contract executed; state updated; funds/resources transferred; DLT block committed
BRAND_STANDARDS_CHECK	Periodic compliance evaluation	DAO governance → member org	OrgDNA version validated; breach detected if out of compliance; cure period started
RESOURCE_TRANSFER	Inter-org resource or data exchange	Source org → destination org	Asset/data transferred; ownership recorded on DLT; both orgs' SDOs updated
ORGOS_LEAVE	Org exits DAO	Leaving org + DAO governance	Exit terms executed per smart contract; outstanding obligations settled; membership removed

6.3 DAO Formation Flow

Step	Actor	Action	Technical Effect
1	Founder Org	Open DAO creation wizard in DAO & DLT OrgApp	UI flow: SCR-DAO-02
2	Founder Org	Define DAO name, governance model, initial smart contracts	Smart contract templates compiled
3	Founder Org	Publish OrgDNA as the canonical brand/base package	OrgDNA v1.0 committed to DLT genesis block
4	Founder Org	Invite member OrgOS instances	Invitation sent via DAO event bus; each invitee reviews smart contract terms
5	Each Member	Accept DAO invitation (multi-sig)	ORGOS_JOIN transaction committed to DLT; member node joins consensus
6	All Members	DAO governance goes live	Smart contracts active; DLT consensus running; inter-org event bus connected
7	Ongoing	Smart contracts execute on trigger events	Royalties, brand checks, governance votes execute automatically

6.4 Scaling Models

Scaling Model	Structure	DAO Config	Use Case
Horizontal (Franchise)	Parent OrgOS + N independent franchisee OrgOS instances	Hierarchical governance; parent controls OrgDNA updates; royalty smart contracts	Dry cleaning franchise; restaurant chain; service franchise network
Vertical (Group/Holding)	Holding company OrgOS + subsidiary OrgOS instances	Federated governance; consolidated financial reporting across all orgs	Investment holding company; group of companies; venture portfolio
Peer (Cooperative)	N equal-member OrgOS instances with no central authority	Flat governance; majority vote decisions; shared treasury	Worker cooperative; consortium; industry body

Supply Chain DAO	Buyer OrgOS + supplier OrgOS instances	Contract-governed; SLA enforcement via smart contracts; shared logistics data	Supplier network; B2B supply chain; procurement consortium

Part VII: Deployment & Infrastructure

7.1 OrgOS Deployment Topologies

Topology	Description	Suitable For
Shared Cloud (SaaS)	Multiple OrgOS instances on shared UME infrastructure; tenant isolation via org_id column and RBAC	Small-medium businesses; standard plans; fastest time to value
Dedicated Cloud	Single-tenant OrgOS instance on dedicated cloud infrastructure; full isolation	Enterprise; regulated industries; data sovereignty requirements
On-Premises	OrgOS deployed to customer-owned infrastructure; UME factory manages via agent	Government; financial services; maximum data control
Hybrid	Core OrgKernel on-premises; specific OrgApps (e.g., Analytics, AI Agents) on cloud	Enterprises with legacy infrastructure; phased cloud migration
DAO Node	Each member OrgOS is a node in a federated network; must be reachable for DLT consensus	Any DAO member organisation; requires fixed endpoint for DLT

7.2 Multi-OrgOS Management (Factory Layer)

The UME Factory layer manages the lifecycle of all provisioned OrgOS instances. It provides:

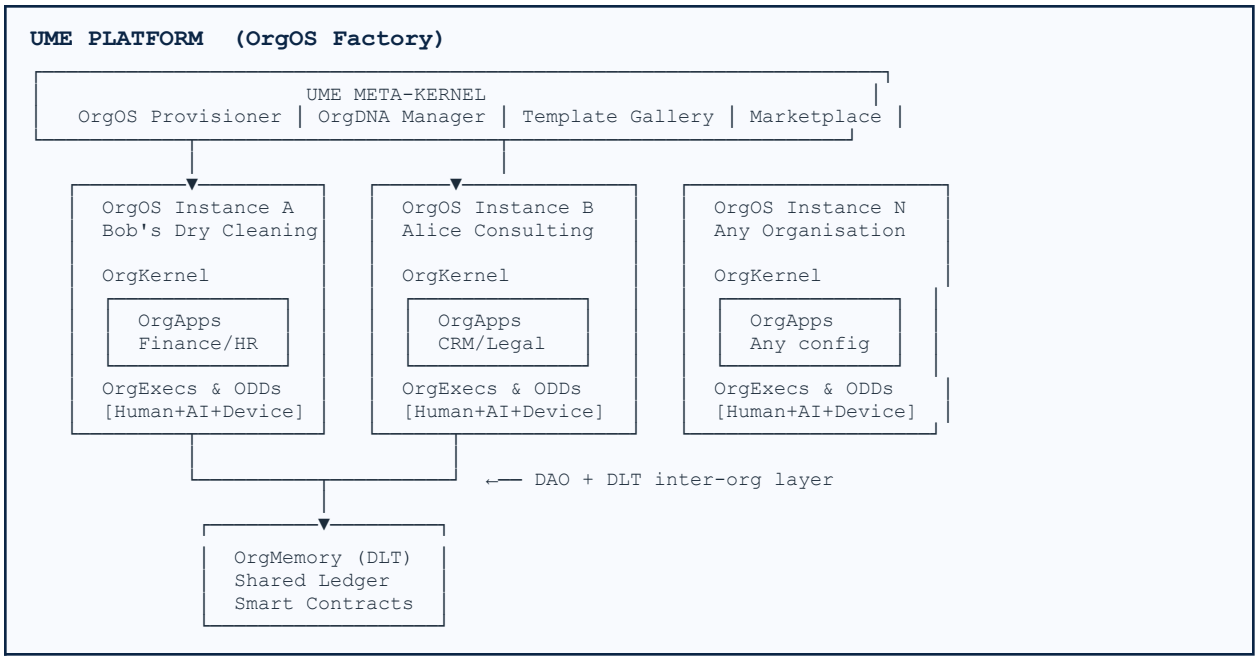
Factory Capability	Description
OrgOS Provisioner	Provisions new OrgOS instances from OrgDNA templates; handles infrastructure allocation, database setup, and initial OrgApp deployment
OrgOS Monitor	Monitors the health, performance, and compliance of all managed OrgOS instances; surfaces issues to the factory administrator
OrgDNA Repository	Maintains a version history of all OrgDNA packages for all managed organisations; enables rollback and diff comparison
OrgOS Upgrade Manager	Applies OrgApp version updates and ODD template updates across one or many OrgOS instances with configurable rollout strategies
Billing & Metering	Tracks OrgApp usage, OrgExec counts, storage consumption, and DLT transaction volume for billing purposes
Template Gallery	Curated library of OrgDNA templates for various industries and organisation types; submitted by community and UME team

Appendix A: Complete C4 Architecture Diagrams

A.1 System Context (Level 0) — OrgOS Factory

OrgOS Factory — System Context Diagram (C4 Level 0)

The UME platform acts as an OrgOS Factory: a meta-operating-system that produces, configures, and maintains Organisation Operating Systems for any organisation.



A.2 Container Diagram (Level 1) — Single OrgOS Instance

Container	Technology	Responsibility
ume_server	Rust/Axum	HTTP REST API; WebSocket; auth middleware; request routing to OrgKernel
OrgKernel	Rust	Composition root; all subsystems; OrgApp lifecycle; RBAC; event bus
OrgApp Fleet	Rust (42+ OrgApps)	Domain business logic; data models; event emission; ODD integration
ODDBus	Rust	ODD contract registry; action validation; constraint enforcement; violation logging
SDOEngine	Rust + LLM hook	Digital twin state aggregation; topology graph; OrgSimulator
OrgDNA Store	PostgreSQL + S3	Versioned OrgDNA packages; config history; snapshot storage
DAOClient / DLTClient	Rust + consensus lib	Inter-org communication; DLT block submission; smart contract execution
Java Client Suite	Java/JavaFX + Bazel	GUI desktop client; TUI watch mode; batch CLI commands
PostgreSQL	PostgreSQL 15+	Primary transactional store; all OrgApp domain data; audit chain

Redis	Redis 7+	Shared cache; session store; real-time dashboard data

Appendix B: ODD Template Library

ODD Template ID	Applies To	Key Capabilities	Key Constraints
FTE-ODD-v2	Full-time human employees	Read/write own OrgApp data; approve own tasks; request leave; view own payslips	No access to payroll of others; no admin changes; no financial postings without approval
EXEC-ODD-v2	Executive / owner OrgExec	Full read on all OrgApps; approve financial postings; manage ODD contracts; export OrgDNA	Cannot modify audit records; dual-control for transactions >$10K
AI-AGENT-ODD-v1	AI agent OrgExecs	Read: configured data sources; Write: configured output targets; Notify: configured channels	Rate-limited; working hours only by default; escalate on ambiguity; all actions audited real-time
IOT-MACHINE-ODD-v1	OrgCPU: IoT devices / machines	Report sensor readings; update own status; trigger configured alerts	Read-only on org data; no financial access; no HR access; status updates only
CONTRACTOR-ODD-v1	External contractor OrgExec	Read/write scoped to assigned project data only; no HR or finance access	Project-scoped only; auto-expires on contract end date; no data export
FRANCHISE-ODD-v1	Franchisee org (DAO member)	Operate full OrgOS; join DAO governance votes; receive OrgDNA updates	Royalty smart contract auto-executes; brand OrgDNA must stay within N minor versions of parent

Appendix C: New Module Data Models (v2.0.0)

Model	Fields	Module
OrgDNA	{ id, org_id, version: SemanticVersion, kernel_config: KernelConfig, app_registry: Vec<AppConfig>, odd_library: Vec<ODD>, rbac_packs: Vec<RbacPack>, coa: ChartOfAccounts, compliance_packs: Vec<CompliancePack>, kpi_definitions: Vec<KpiDef>, created_at, published_by }	OrgDNA Manager
OrgExecRecord	{ id: OrgExecId, display_name, exec_type: OrgExecType, odd_id: OddId, odd_version, status: ExecStatus, created_at, last_active }	OrgExec Manager
OddContract	{ id: OddId, version, exec_id, capabilities: Vec<OddCapability>, constraints: Vec<OddConstraint>, data_access: Vec<DataAccessGrant>, escalation_rules: Vec<EscalationRule>, audit_config: AuditConfig, effective_from, expires_at: Option<DateTime> }	ODD Manager
SDONode	{ id: NodeId, org_id, node_type: NodeType, display_name, status: NodeStatus, cpu_pct: f32, last_event: EventEnvelope, data_flows: Vec<DataFlowEdge> }	SDO Engine

DAOMember	{ org_id, org_name, membership_status, consensus_node_url, voting_weight, joined_at, orgdna_version }	DAO Module
DLTBlock	{ block_num: u64, timestamp, tx_type: DLTTxType, from_org: OrgId, to_org: Option<OrgId>, payload_hash: String, signatories: Vec<OrgId>, consensus_state }	DLT Module
SmartContract	{ id, dao_id, contract_type, parties: Vec<OrgId>, terms: Vec<ContractTerm>, trigger: ContractTrigger, status: ContractStatus, deployed_block: u64, last_executed: Option<DateTime> }	DAO Module

Appendix D: Updated Glossary

New Concepts & Terminology (v2.0.0)

The following new concepts were introduced in v2.0.0 of the UME platform. All prior documents that reference "UME platform", "kernel", "modules", and "device drivers" should be read in conjunction with these updated definitions.

Term	Symbol	Definition
OrgOS	OrgOS	A configured, running Organisation Operating System produced by the UME platform. Every organisation provisioned on UME receives a dedicated OrgOS instance, which is the combination of an OrgKernel and its set of active OrgApps.
OrgOS Factory	UME Platform	The UME platform itself is the factory that produces, configures, deploys, and maintains OrgOS instances. UME is a meta-operating-system: an operating system that creates and manages operating systems.
OrgKernel	OrgKernel	The running instance of the UME kernel for a specific organisation. The OrgKernel is the composition root, resource manager, and supervisor for all OrgApps within a single organisation's OrgOS.
OrgApp	OrgApp	A modular application running within an OrgOS, implementing a specific organisational capability (Finance, HR, Legal, etc.). Previously called "organisation modules". OrgApps are the user-facing applications that manage and run the organisation.
OrgExec	OrgExec	An executor of work within an OrgOS: a human employee, AI agent, hybrid human-AI system, or any autonomous actor that receives tasks and executes them on behalf of the organisation. OrgExecs are governed by ODD contracts.
OrgCPU	OrgCPU	A digital-only executor within an OrgOS: machines, IoT devices, sensors, 3rd-party platforms, manufacturing equipment, POS terminals, and any non-human device that executes work. OrgCPUs are governed by ODD device driver contracts.
ODD	OrgDeviceDriver (ODD)	A formal contract that defines how a specific OrgExec or OrgCPU operates within the OrgOS. Specifies capabilities (what it can do), constraints (what it must not do), data access rights, rate limits, escalation triggers, and audit requirements. ODDs are the primary mechanism for governing all actors in an OrgOS.
OrgDNA	OrgDNA	The exportable, versionable master configuration package of a fully configured OrgOS. OrgDNA captures the OrgKernel config, all active OrgApps and their settings, ODD templates, RBAC policies, chart of accounts, KPIs, and jurisdiction compliance packs. OrgDNA can be cloned, forked, shared with franchisees, or published as a template.
SDO	Software Defined Organisation (SDO)	The digital twin of a physical organisation. An SDO is the complete runtime representation of an organisation within the UME platform — all running OrgApps, all OrgExecs, all OrgCPUs, all data flows, and all organisational state. Users operate at the SDO layer; the physical organisation is realised through the digital twin.

OrgRuntime	Organisation Runtime	The live execution of an OrgOS: the combination of the OrgKernel, all running OrgApps, all active OrgExecs, all connected OrgCPUs, and all live organisational data. The OrgRuntime realises organisation outcomes in real time.
OrgMemory	OrgMemory / DLT	The persistent, immutable record of all significant organisational events and inter-organisation transactions. Within a single OrgOS, OrgMemory is maintained by the OrgKernel audit chain. Across multiple OrgOS instances (DAO), OrgMemory is a Distributed Ledger (DLT) shared among all member organisations.
DAO	Decentralised Autonomous Organisation (DAO)	A set of interconnected, independent OrgOS instances that cooperate under a shared governance model. DAOs are formed through the UME DAO module, which provides smart contract governance, distributed ledger infrastructure, shared OrgDNA versioning, and inter-organisation event routing.
DLT	Distributed Ledger Technology (DLT)	The inter-organisation ledger that records all cross-OrgOS transactions, smart contract executions, OrgDNA updates, and governance votes for a DAO. The DLT is the shared OrgMemory of a DAO — immutable, cryptographically verified, and replicated across all member OrgOS nodes.
Smart Contract	Smart Contract	An automatically executable agreement between two or more OrgOS instances within a DAO, stored on the DLT. Smart contracts govern resource transfers (royalties, payments), brand standards enforcement, governance decisions, and any other inter-organisation obligation.
OrgExec Type	OrgExec Classification	Human (natural persons employed/contracted), AI Agent (LLM-powered or rule-based autonomous agent), Hybrid (human-AI collaboration system), or Physical Agent (OrgCPU: device/machine/sensor).